

Overview of Momentum optimization

Subject: Momentum optimization

1.

where the parameter w which minimizes $Q(w)$ is to be estimated, η is a step size (sometimes called the learning rate in machine learning) and α is an exponential decay factor between 0 and 1 that determines the relative contribution of the current gradient and earlier gradients to the weight change. The name momentum stems from an analogy to momentum in physics: the weight vector w , thought of as a particle traveling through parameter space, incurs acceleration from the gradient of the loss ("force"). Unlike in classical stochastic gradient descent, it tends to keep traveling in the same direction, preventing oscillations. Momentum has been used successfully by computer scientists in the training of artificial neural networks for several decades. The momentum method is closely related to underdamped Langevin dynamics, and may be combined with simulated annealing. In mid-1980s the method was modified by Yurii Nesterov to use the gradient predicted at the next point, and the resulting so-called Nesterov Accelerated Gradient was sometimes used in ML in the 2010s.

2.

$$\max_{k=0, \dots, \lfloor T/\eta \rfloor} |\mathbb{E}[g(w_k)] - \mathbb{E}[g(W_{k\eta})]| \leq C\eta^2. \quad \{\displaystyle \max_{k=0, \dots, \lfloor T/\eta \rfloor} |\mathbb{E}[g(w_k)] - \mathbb{E}[g(W_{k\eta})]| \leq C\eta^2.\}$$

3.

$\bar{w} = \frac{1}{t} \sum_{i=0}^{t-1} w_i$. When optimization is done, this averaged parameter vector takes the place of w .

4.

Adam (short for Adaptive Moment Estimation) is a 2014 update to the RMSProp optimizer combining it with the main feature of the Momentum method. In this optimization algorithm, running averages with exponential forgetting of both the gradients and the second moments of the gradients are used. Given parameters $w(t)$ and a loss function $L(t)$, where t indexes the current training iteration (indexed at 1), Adam's parameter update is given by:

5.

$Q(w) = \frac{1}{n} \sum_{i=1}^n Q_i(w) = \frac{1}{n} \sum_{i=1}^n (m(w; x_i) - y_i)^2$, where $m(w; x_i)$ is the predictive model (e.g., a deep neural network) the objective's structure can be exploited to estimate 2nd order information using gradients only. The resulting methods are simple and often effective

6.

$$Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n (\hat{y}_i - y_i)^2 = \sum_{i=1}^n (w_1 + w_2 x_i - y_i)^2.$$

$$\{ \displaystyle Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n \left(\hat{y}_i - y_i \right)^2 = \sum_{i=1}^n \left(w_1 + w_2 x_i - y_i \right)^2 .$$

7.

where the parameter w that minimizes $Q(w)$ is to be estimated. Each summand function Q_i is typically associated with the i -th observation in the data set (used for training). In classical statistics, sum-minimization problems arise in least squares and in maximum-likelihood estimation (for independent observations). The general class of estimators that arise as minimizers of sums are called M-estimators. However, in statistics, it has been long recognized that requiring even local minimization is too restrictive for some problems of maximum-likelihood estimation. Therefore, contemporary statistical theorists often consider stationary points of the likelihood function (or zeros of its derivative, the score function, and other estimating equations). The sum-minimization problem also arises for empirical risk minimization. There, $Q_i(w)$ is the value of the loss function at i -th example, and $Q(w)$ is the empirical risk. When used to minimize the above function, a standard (or "batch") gradient descent method would perform the following iterations:

8.

$$m_w(t) := \beta_1 m_w(t-1) + (1 - \beta_1) \nabla_w L(t-1)$$

$$\{ \displaystyle m_w(t) := \beta_1 m_w(t-1) + \left(1 - \beta_1 \right) \nabla_w L(t-1)$$

9.

$$w^{\text{new}} = w^{\text{old}} + \frac{\eta}{1 + \eta \left\| x_i \right\|^2} \left(y_i - x_i' w^{\text{old}} \right) x_i.$$

$$\{ \displaystyle w^{\text{new}} = w^{\text{old}} + \frac{\eta}{1 + \eta \left\| x_i \right\|^2} \left(y_i - x_i' w^{\text{old}} \right) x_i .$$

10.

Sign-based stochastic gradient descent Even though sign-based optimization goes back to the aforementioned Rprop, in 2018 researchers tried to simplify Adam by removing the magnitude of the stochastic gradient from being taken into account and only considering its sign. This results in a significantly lower communication cost of transferring gradients from workers to the parameter server. In this sense, it serves to better compress the gradient information, while having comparable convergence to standard SGD.

11.

Notable applications Stochastic gradient descent is a popular algorithm for training a wide range of models in machine learning, including (linear) support vector machines, logistic regression (see, e.g., Vowpal Wabbit) and graphical models. When combined with the backpropagation algorithm, it is the de facto standard algorithm for training artificial neural networks. Its use has been also reported in the Geophysics community, specifically to applications of Full Waveform Inversion (FWI). Stochastic gradient descent competes with the L-BFGS algorithm, which is also widely used. Stochastic gradient descent has been used since at least 1960 for training linear regression models, originally under the

name ADALINE. Another stochastic gradient descent algorithm is the least mean squares (LMS) adaptive filter.

12.

Further proposals include the momentum method or the heavy ball method, which in ML context appeared in Rumelhart, Hinton and Williams' paper on backpropagation learning and borrowed the idea from Soviet mathematician Boris Polyak's 1964 article on solving functional equations. Stochastic gradient descent with momentum remembers the update Δw at each iteration, and determines the next update as a linear combination of the gradient and the previous update: